



# Jetpack Compose



Un cambio de paradigma



# ¿Qué es Jetpack Compose?

---

Un nuevo toolkit para crear interfaces de usuario nativas en Android.

Declarativo: Describe cómo debe verse la UI, no cómo construirla.

Menor código: Simplifica el desarrollo de UI.

Interoperable: Se integra con vistas tradicionales.

Rendimiento: Ofrece un rendimiento fluido y eficiente.

# ¿Por qué usarlo?

---

Más fácil de leer y mantener: Código más conciso y limpio.

Mayor productividad: Menos código, menos errores.

Experiencia de usuario mejorada: Interfaces más fluidas y responsivas.

Integración con Material Design: Crea aplicaciones visualmente atractivas.

| Característica        | XML                                                                               | Jetpack Compose                                                   |
|-----------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Sintaxis              | Verbosa, declarativa y con demasiado texto                                        | Declarativa, programática y concisa                               |
| Lenguaje              | Java o Kotlin                                                                     | Kotlin únicamente                                                 |
| Estructura del código | Cada layout tiene un archivo XML separado, a veces complejo y difícil de mantener | Código Kotlin en línea dentro de tu código, más fácil de mantener |
| Reactividad           | No reactivo                                                                       | Reactivo                                                          |
| Curva de aprendizaje  | Fácil de aprender                                                                 | Más fácil de aprender que XML                                     |
| Personalización       | Personalizable                                                                    | Altamente personalizable                                          |
| Adopción              | Ampliamente utilizado en proyectos antiguos/anteriores                            | Ampliamente utilizado en proyectos recientes                      |
| Comunidad             | Extensa                                                                           | Creciendo rápidamente                                             |
| Era                   | Antiguo                                                                           | Moderno                                                           |
| Propietario           | W3C                                                                               | Google Inc.                                                       |

# Lo que aprenderemos

---

Fundamentos: Componibles, modificadores, layouts básicos.

Interfaces complejas: ConstraintLayout, listas, navegación.

Temas y estilos: Personaliza la apariencia de tu app.

Estados y efectos secundarios: Maneja cambios en la UI.

Avanzado: Animaciones, interoperabilidad, testing.

# Conceptos básicos de Compose

---

# Componibles:

---

Son funciones de Kotlin que construyen la UI.

Ejemplo sencillo:

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hola, $name!")
}
```

# Declaratividad en Jetpack Compose

---

¿Qué significa declaratividad en el contexto de Jetpack Compose?

En términos simples, la declaratividad en Jetpack Compose significa **describir cómo queremos que se vea nuestra interfaz de usuario en un momento dado, en lugar de especificar paso a paso cómo construirla.**

Es como darle a un ordenador una receta de un pastel: le dices los ingredientes y los pasos generales, y ella se encarga de mezclar, hornear y decorar el pastel.



---

## Característica

## XML

## Jetpack Compose

Enfoque

Imperativo (cómo hacer algo)

Declarativo (qué queremos obtener)

Sintaxis

Verbosa, basada en atributos

Concisa, basada en funciones de Kotlin

Reactividad

Manual

Automática

# Modificadores

---

Personalizan los composables.

```
Text(  
    text = "Hola",  
    fontSize = 24.sp,  
    color = Color.Blue  
)
```

---

Los modificadores son funciones que se aplican a un composable para modificar su apariencia, comportamiento o posición dentro de la jerarquía de la UI.

Son como adornos que le agregas a un elemento para personalizarlo a tu gusto.

# ¿Cómo funcionan los modificadores?

---

**Composición:** Los modificadores se encadenan uno tras otro para crear una secuencia de modificaciones.

**Orden:** El orden en que se aplican los modificadores es importante, ya que los modificadores posteriores pueden anular los efectos de los anteriores.

**Inmutabilidad:** Los modificadores son inmutables, lo que significa que no se pueden modificar después de ser creados.

# Tipos de modificadores:

---

**Modificadores de layout:** Controlan la posición y el tamaño de un composable dentro de un layout. Ejemplos: `padding`, `size`, `fillMaxSize`, `wrapContentSize`, `constrainAs`.

**Modificadores de interacción:** Hacen que un composable responda a las interacciones del usuario. Ejemplos: `clickable`, `draggable`, `pointerInput`.

**Modificadores de apariencia:** Afectan la apariencia visual de un composable. Ejemplos: `background`, `border`, `clip`, `alpha`.

**Modificadores de tema:** Aplican estilos y temas a un composable. Ejemplos: `color`, `typography`.

# Formas de declarar modificadores:

---

# Implícita

---

En esta sintaxis, el nombre del modificador se utiliza como una propiedad del composable. Es la forma más común y concisa de aplicar modificadores.

```
Text(  
    text = "Hola, mundo!",  
    padding = 16.dp,  
    background = Color.LightGray  
)
```

# Explícita

---

Esta sintaxis es más explícita y puede ser útil cuando se necesitan aplicar muchos modificadores o cuando se crean modificadores personalizados.

```
Text(  
    text = "Hola, mundo!",  
    modifier = Modifier  
        .padding(16.dp)  
        .background(Color.LightGray)  
)
```



# Modificadores personalizados:

---

Puedes crear tus propios modificadores para encapsular una secuencia común de modificadores o para crear comportamientos personalizados.

```
fun Modifier.myCustomModifier() = this
    .padding(16.dp)
    .background(Color.LightGray)
    .clickable { /* Acción al hacer clic */ }

Text(
    text = "Hola, mundo!",
    modifier = Modifier.myCustomModifier()
)
```

# Cuidado!!

---

"text" no es un modificador en Jetpack Compose.

**Sintaxis similar:** Tanto los modificadores como los parámetros de los composables se escriben dentro de las llaves de la función composable y se separan por comas. Esto puede llevar a pensar que todos los elementos dentro de las llaves son modificadores.

# Layouts Básicos en Jetpack Compose

---

Los layouts básicos en Jetpack Compose proporcionan las herramientas fundamentales para estructurar tus interfaces de usuario. Estos layouts son **composables** que te permiten organizar tus elementos de forma lineal, en una cuadrícula o de manera más flexible.

# Column

---

Organiza elementos **verticalmente**: Coloca los elementos uno debajo del otro.

```
Column {  
    Text(text = "Elemento 1")  
    Button(onClick = { /* Acción al hacer clic */ }) {  
        Text(text = "Botón")  
    }  
}
```

# Row

---

Organiza elementos **horizontalmente**: Coloca los elementos uno al lado del otro.

```
Row {  
    Icon(imageVector = Icons.Default.Home, contentDescription = "Inicio")  
    Spacer(modifier = Modifier.weight(1f))  
    Icon(imageVector = Icons.Default.Search, contentDescription = "Buscar")  
}
```

# Box

---

Organiza elementos en un contenedor flexible: Permite **superponer** elementos y controlar su alineación.

```
Box {  
    Text(text = "Texto de fondo")  
    Button(  
        onClick = { /* Acción al hacer clic */ },  
        modifier = Modifier.align(Alignment.Center)  
    ) {  
        Text(text = "Botón")  
    }  
}
```

# Modificadores de layout y sus usos:

---

**padding**: Agrega espacio alrededor de un elemento.

**size**: Establece el tamaño fijo de un elemento.

**fillMaxSize**: Hace que un elemento ocupe todo el espacio disponible.

**wrapContentSize**: Hace que un elemento se ajuste al tamaño de su contenido.

**weight**: Distribuye el espacio disponible entre los elementos hijos de un Row o Column.

# ¿Cómo lograr una distribución proporcional con Box?

---

Aunque **Box** no tiene **weight**, puedes lograr efectos similares utilizando combinaciones de otros modificadores y layouts:

- **ConstraintLayout**: Para distribuciones más complejas y precisas, ConstraintLayout es la mejor opción. Te permite definir relaciones entre elementos de forma visual y establecer proporciones entre ellos.
- **Combinación de Box y Row/Column**: Puedes anidar un Row o Column dentro de un Box para lograr una distribución proporcional de los elementos.
- **Modificadores de tamaño**: Utilizar modificadores como **fillMaxWidth**, **fillMaxHeight** y **weight** dentro de un Row o Column anidado dentro de un Box para controlar el tamaño relativo de los elementos.



# Interfaces Complejas en Jetpack Compose

---

# ConstraintLayout: Creando diseños precisos y flexibles

---

## ¿Cuándo usarlo?

Cuando necesitas crear diseños complejos con relaciones precisas entre elementos, como aquellos que se encuentran en diseños de cuadrícula o diseños que deben adaptarse a diferentes tamaños de pantalla.

```
ConstraintLayout {  
    val (button, text) = createRefs()  
  
    Button(  
        onClick = { /* Acción al hacer clic */ },  
        modifier = Modifier.constrainAs(button) {  
            top.linkTo(parent.top)  
            start.linkTo(parent.start)  
        }  
    )  
  
    Text(  
        text = "Hola, mundo!",  
        modifier = Modifier.constrainAs(text) {  
            top.linkTo(button.bottom)  
            start.linkTo(parent.start)  
        }  
    )  
}
```

# ¿Qué es createRefs?

---

`createRefs` es una función de utilidad que se utiliza dentro de un `ConstraintLayout` para crear `referencias` a los elementos que deseas posicionar.

Estas referencias se almacenan en variables y se utilizan posteriormente para definir las restricciones.

# Listas: LazyColumn y LazyRow

---

**Para listas largas:** Utilizados para mostrar grandes cantidades de datos de forma eficiente.

**Virtualización:** Solo se renderizan los elementos visibles en la pantalla, mejorando el rendimiento.

**Personalización:** Puedes personalizar cada elemento de la lista y agregar interacciones.

# LazyColumn

---

Organiza elementos **verticalmente**: Ideal para listas que se desplazan hacia arriba y hacia abajo.

```
LazyColumn {  
    items(items = listOf("Item 1", "Item 2", "Item 3")) { item ->  
        Text(text = item)  
    }  
}
```

# LazyRow

---

Organiza elementos **horizontalmente**: Ideal para listas que se desplazan hacia la izquierda y hacia la derecha.

```
LazyRow {  
    items(items = listOf(ImageItem1, ImageItem2, ImageItem3)) { item ->  
        Image(painter = item.painter, contentDescription = item.contentDescripti  
    }  
}
```

# Ejemplos más avanzados

---



# Listas con encabezados y pies de página:

---

```
LazyColumn {  
    item { Text("Encabezado") }  
    items(items = items) { item ->  
        // Contenido del elemento  
    }  
    item { Text("Pie de página") }  
}
```

# Listas con carga diferida:

---

```
LazyColumn {  
    items(  
        count = items.size,  
        key = { it },  
        onItemsLoaded = {  
            // Cargar más elementos si es necesario  
        }  
    ) { index ->  
        // Contenido del elemento  
    }  
}
```

# Personalización de elementos en LazyColumn y LazyRow

---

# Key (Clave)

---

**Propósito:** Cada elemento en una lista debe tener una clave única para que Compose pueda identificar y actualizar correctamente los elementos cuando los datos cambian. Esto es crucial para animaciones, reordenamientos y optimizaciones de rendimiento.

**Tipos de claves:** Puedes usar cualquier tipo de dato inmutable como clave, como un ID, un índice o un objeto de datos.

# Item entero

---

```
LazyColumn {  
    items(items = listOf("Elemento 1", "Elemento 2", "Elemento 3")) { item ->  
        // Usando el propio elemento como clave (si es único)  
        Text(text = item)  
    }  
}
```

# Solo la ID

---

Utilizar un índice en lugar del elemento completo como clave en LazyColumn y LazyRow es una práctica común y altamente recomendada, especialmente cuando trabajas con grandes conjuntos de datos o cuando los elementos de tu lista son objetos complejos.

```
val items = listOf("Elemento 1", "Elemento 2", "Elemento 3")

LazyColumn {
    items(items.indices) { index ->
        Text(text = items[index])
    }
}
```

# ItemProvider

---

**Definición:** Define cómo se crea cada elemento de la lista a partir de los datos proporcionados.

```
LazyColumn {  
    items(items = listOf(User("Alice", 25), User("Bob", 30))) { user ->  
        Card {  
            Text(text = user.name)  
            Text(text = "Edad: ${user.age}")  
        }  
    }  
}
```

# Indicadores de carga

**Visualización del estado:** Muestra un indicador de carga mientras se cargan los datos, mejorando la experiencia del usuario.

**Integración con State:** Utiliza `remember` y `mutableStateOf` para gestionar el estado de carga.

```
var isLoading by remember { mutableStateOf(true) }  
LazyColumn {  
    items(items = items) { item ->  
        // Contenido del elemento  
    }  
    item {  
        if (isLoading) {  
            CircularProgressIndicator()  
        }  
    }  
}
```



# Separadores

---

Mejora de la legibilidad: Agrega separadores entre los elementos para mejorar la legibilidad de la lista.

```
LazyColumn {  
    items(items = items) { item ->  
        Text(text = item)  
        Divider()  
    }  
}
```

# Características adicionales de LazyColumn y LazyRow

---

# LazyVerticalGrid y LazyHorizontalGrid

---

Estos componentes son ideales para crear layouts en forma de cuadrícula. Imagina una galería de imágenes o una lista de productos en un catálogo en línea.

- **LazyVerticalGrid**: Crea una cuadrícula donde los elementos se organizan en filas y columnas de forma vertical.
- **LazyHorizontalGrid**: Similar al anterior, pero organiza los elementos en filas y columnas de forma horizontal.

---

En este ejemplo, creamos una cuadrícula con 2 columnas. Cada elemento de la lista se colocará en la siguiente celda disponible.

```
LazyVerticalGrid(cells = GridCells.Fixed(2)) {  
    items(items = listOf(ImageItem1, ImageItem2, ImageItem3)) { item ->  
        Image(painter = item.painter, contentDescription = item.contentDescription)  
    }  
}
```

# LazyListState

---

Este estado te permite controlar el desplazamiento de la lista. Puedes obtener la **posición actual**, verificar si se ha llegado al **inicio** o al **final**, y realizar otras acciones relacionadas con el **desplazamiento**.

```
val listState = rememberLazyListState()

LazyColumn(state = listState) {
    // ...
}
```

# rememberLazyListState

---

Esta función composable se utiliza para recordar el estado de desplazamiento entre composiciones. Esto es útil cuando necesitas mantener el estado de desplazamiento al navegar entre pantallas o al cambiar la configuración de la lista.

# Acciones con LazyListState:

---

**scrollToItem**: Desplazar la lista a un elemento específico.

**scrollToTop**: Desplazar la lista al principio.

**scrollToBottom**: Desplazar la lista al final.

**isScrollInProgress**: Verificar si la lista está siendo desplazada.

# 1. Desplazar la lista a un elemento específico:

```
val listState = rememberLazyListState()

LazyColumn(state = listState) {
    items(items = listOf("Elemento 1", "Elemento 2", "Elemento 3")) { item ->
        Text(text = item)
    }
}

// Desplazar al segundo elemento cuando se presione un botón
Button(onClick = {
    listState.scrollToItem(index = 1)
}) {
    Text(text = "Ir al segundo elemento")
}
```



## 2. Desplazar la lista al principio o al final:

---

```
Button(onClick = {  
    listState.scrollToItem(index = 0) // Desplazar al inicio  
}) {  
    Text(text = "Ir al principio")  
}  
  
Button(onClick = {  
    listState.scrollToItem(index = items.lastIndex) // Desplazar al final  
}) {  
    Text(text = "Ir al final")  
}
```

### 3. Verificar si la lista está siendo desplazada:

---

```
val listState = rememberLazyListState()

LazyColumn(state = listState) {
    // ...
}

if (listState.isScrollInProgress) {
    // Mostrar un indicador de carga mientras se desplaza
    CircularProgressIndicator()
}
```

## 4. Animar el desplazamiento:

---

```
Button(onClick = {  
    listState.animateScrollToItem(index = 0)  
}) {  
    Text(text = "Ir al principio con animación")  
}
```

# Ejercicios básicos

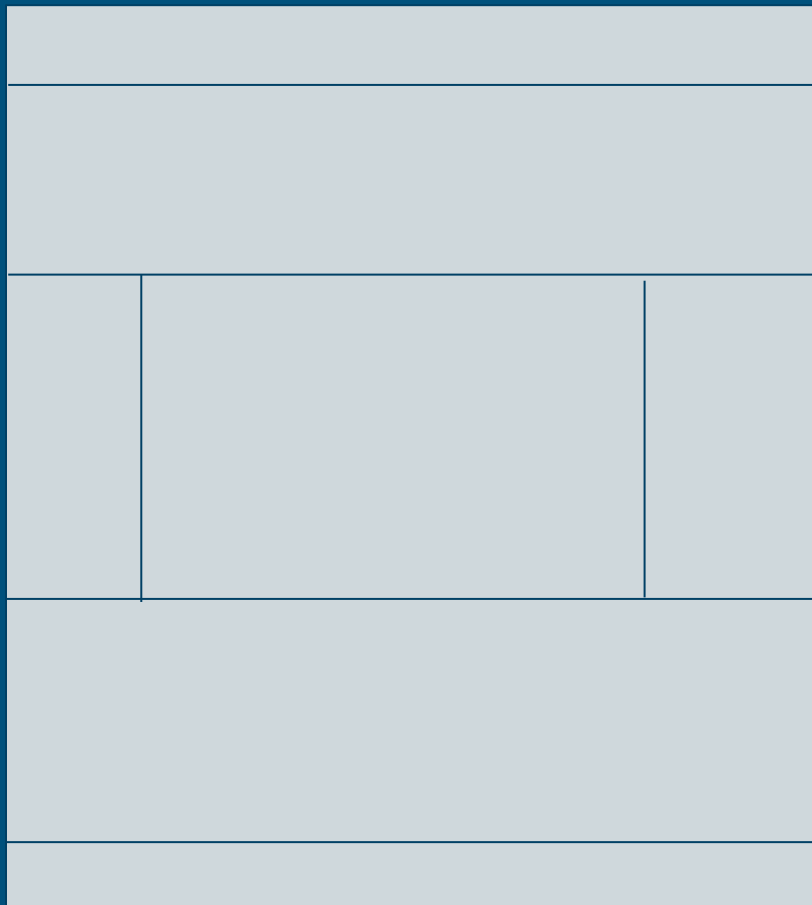
---

1. **Hola, Mundo! con modificadores:** Crea un simple composable que muestre el texto "Hola, Mundo!" con un fondo de color y un padding personalizado.
2. **Lista de tareas:** Crea una lista vertical de tareas utilizando LazyColumn. Cada tarea debe ser un elemento clickable que al ser presionado muestre un mensaje de "Tarea completada".
3. **Cuadrícula de imágenes:** Utiliza LazyVerticalGrid para crear una cuadrícula de imágenes. Cada imagen debe tener un título y al hacer clic en ella, se debe mostrar una pantalla de detalle.

# Ejercicio 4

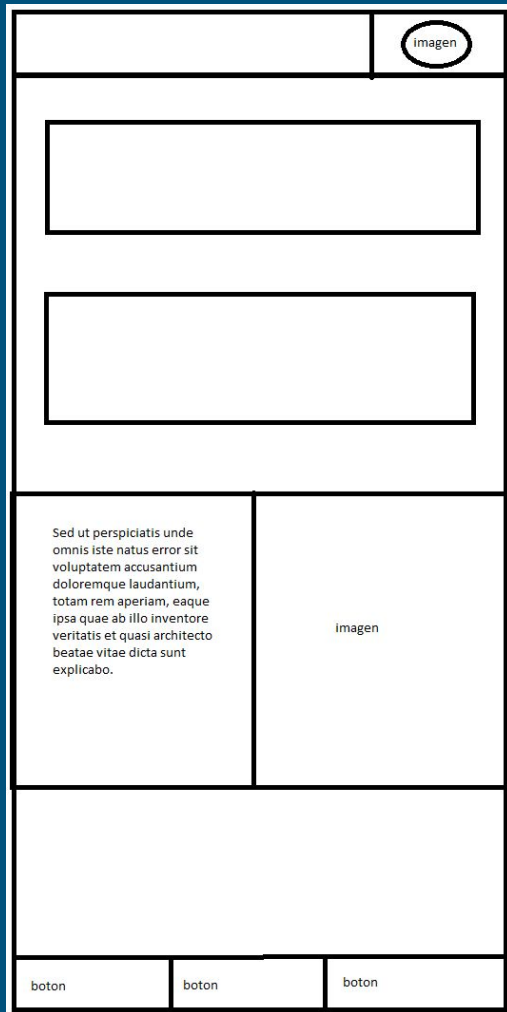
---

Recrea esto con las herramientas que quieras vistas anteriormente en Compose exclusivamente.



# Ejercicio 5

Hazlo lo mejor  
que puedas....



# Navegación

---

# Scaffold

---

El **Scaffold** es un componente fundamental en Jetpack Compose que proporciona una estructura básica y consistente para tus aplicaciones Android. Imagina el Scaffold como un andamio que te permite construir tu aplicación de manera organizada y eficiente.



El Scaffold es un composible que define la estructura general de una pantalla, incluyendo elementos como:

---

**AppBar:** La barra superior que puede contener un título, acciones y otros elementos.

**BottomAppBar:** La barra inferior que puede contener botones de navegación o acciones.

**FloatingActionButton:** Un botón flotante que se coloca generalmente en la esquina inferior derecha.

**Snackbar:** Un mensaje breve que aparece en la parte inferior de la pantalla para mostrar notificaciones.

**Drawer:** Un menú lateral que se despliega desde un borde de la pantalla.

**Body:** El contenido principal de la pantalla.

# ¿Por qué usar Scaffold?

---

**Estructura consistente:** Proporciona una estructura básica para todas tus pantallas.

**Facilidad de uso:** Simplifica la creación de pantallas complejas.

Integración con **Material Design:** Se ajusta a las guías de diseño de Material Design.

**Personalización:** Puedes personalizar cada elemento del Scaffold para adaptarlo a tus necesidades.

```
Scaffold(  
  topBar = {  
    TopAppBar(title = { Text("Mi Aplicación") })  
  },  
  floatingActionButton = {  
    FloatingActionButton(onClick = { /* Acción al hacer clic */ }) {  
      Icon(Icons.Default.Add, contentDescription = "Agregar")  
    }  
  },  
  content = {  
    Text(text = "Contenido principal de la pantalla")  
  }  
)
```

# Navigation Component

---

El Navigation Component te ayuda a gestionar la navegación entre diferentes pantallas en tu aplicación. Cuando creas un nuevo proyecto Compose con navegación, se genera una estructura básica que incluye:

- **NavHost**: Un contenedor que alberga los diferentes destinos de navegación.
- **NavGraph**: Define las rutas y destinos de navegación.
- **Composable**: Cada pantalla es un composable que se asigna a un destino específico en el NavGraph.

# Recurso

---

<https://developer.android.com/guide/navigation?hl=es-419>



**Importante:** Ya sea que uses Compose, vistas o un framework de IU personalizado, estos conceptos siempre se aplican cuando se implementa la navegación. Sin embargo, las formas específicas en que se usan pueden diferir.

| Concepto    | Propósito                                                                                                                                                                                                        | Tipo                                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Host        | Un elemento de la IU que contiene el destino de navegación actual. Es decir, cuando un usuario navega por una app, esta esencialmente intercambia destinos dentro y fuera del host de navegación.                | <ul style="list-style-type: none"> <li>• Compose: <code>NavHost</code></li> <li>• Fragmentos: <code>NavHostFragment</code></li> </ul> |
| Gráfico     | Es una estructura de datos que define todos los destinos de navegación dentro de la app y cómo se conectan entre sí.                                                                                             | <code>NavGraph</code>                                                                                                                 |
| Controlador | Es el coordinador central para administrar la navegación entre destinos. El controlador ofrece métodos para navegar entre destinos, controlar vínculos directos, administrar la pila de actividades y mucho más. | <code>NavController</code>                                                                                                            |
| Destino     | Nodo en el gráfico de navegación. Cuando el usuario navega a este nodo, el host muestra su contenido.                                                                                                            | <code>NavDestination</code><br><br>Por lo general, se crea al construir el gráfico de navegación.                                     |
| Ruta        | Identifica de manera inequívoca un destino y los datos que requiere.<br><br>Puedes navegar por las rutas. Las rutas te llevan a destinos.                                                                        | Cualquier tipo de datos serializables.                                                                                                |

# ¿Cómo se define el gráfico de navegación en Compose?

---

En Jetpack Compose, no es necesario definir un archivo XML separado para el gráfico de navegación. A diferencia del enfoque tradicional con XML, Compose utiliza un enfoque declarativo y programático para definir la estructura de la navegación.

# Estructura

---

1. **NavHost**: Es el componente principal que contiene todos los destinos de navegación.
2. **NavGraph**: Define las rutas y destinos de navegación.
3. **Composable**: Cada pantalla es un composable que se asigna a un destino específico en el NavGraph.



# ¿Cómo se usa el navigation en la aplicación?

---

Normalmente, el punto de entrada de una aplicación Compose es un composable que engloba toda la interfaz de usuario. Este composable suele llamarse `@Composable fun MyApp()`. Dentro de este composable, se crea un `NavHost` y se configura el gráfico de navegación.

```
@Composable
fun MyApp() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = "screen_a") {
        composable("screen_a") {
            ScreenA(navController = navController)
        }
        composable("screen_b") {
            ScreenB()
        }
    }
}
```

# Explicación:

---

**rememberNavController()**: Crea un controlador de navegación que se utilizará para navegar entre las diferentes pantallas.

**NavHost**: Es el contenedor principal para la navegación.

**startDestination**: Especifica la pantalla inicial.

**composable**: Define cada pantalla como un composable y lo asigna a una ruta.

# Navegación con scaffold

---

```
@Composable
fun MyScreen() {
    val navController = rememberNavController()
    Scaffold(
        topBar = {
            TopAppBar(title = { Text("Mi Aplicación") })
        },
        bottomBar = {
            BottomNavigation {
                // ... ítems de navegación
            }
        },
        drawerContent = {
            // ... contenido del drawer
        },
        content = {
            NavHost(navController, startDestination = "screen_a") {
                composable("screen_a") {
                    ScreenA(navController = navController)
                }
                composable("screen_b") {
                    ScreenB()
                }
            }
        }
    )
}
```

# NavGraph

---

En Jetpack Compose, el uso de un NavGraph definido en un archivo XML no es obligatorio.

Aunque es una práctica común y útil para proyectos de gran escala, no es un requisito indispensable.

# ¿Cuándo usar un NavGraph XML?

---

**Proyectos grandes y complejos:** Si tienes una aplicación con muchas pantallas y una navegación profunda, un gráfico de navegación XML puede ayudarte a visualizar la estructura de tu aplicación y facilitar el mantenimiento.

**Integración con herramientas de diseño:** Algunas herramientas de diseño pueden generar automáticamente un gráfico de navegación XML, lo que puede agilizar el proceso de diseño.

# En resumen

---

La elección de utilizar un NavGraph XML o definir la navegación en código depende de tus preferencias y de las necesidades de tu proyecto. Ambos enfoques son válidos y tienen sus propias ventajas.



# ¿Cómo crear un NavGraph?

---

1. **Crear un recurso XML:** En el directorio res/navigation, crea un nuevo recurso de tipo "Navigation".
2. **Definir los destinos:** Dentro del archivo XML, define cada pantalla como un destino, especificando su ID y otros atributos como el nombre y los argumentos.
3. **Establecer el destino inicial:** Indica cuál es la primera pantalla que se mostrará cuando se inicie la aplicación.

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/screen_a">

    <fragment
        android:id="@+id/screen_a"
        android:name="com.example.myapp.ScreenA"
        android:label="Screen A" />

    <fragment
        android:id="@+id/screen_b"
        android:name="com.example.myapp.ScreenB"
        android:label="Screen B" >
        <argument
            android:name="userId"
            app:argType="integer" />
    </fragment>
</navigation>
```

# Utilizar el NavGraph en Compose:

---

```
@Composable
fun MyApplication() {
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = "screen_a") {
        composable("screen_a") {
            ScreenA(navController = navController)
        }
        composable("screen_b") {
            val userId = navBackStackEntry.arguments?.getInt("userId") ?: 0
            ScreenB(userId = userId)
        }
    }
}
```

# Explicación

---

**NavHost**: Es el contenedor principal para la navegación.

**NavController**: Se utiliza para navegar entre destinos.

**composable**: Define cada pantalla como un composable.

**startDestination**: Especifica la pantalla inicial.

**navBackStackEntry**: Proporciona información sobre el destino actual y sus argumentos.

# Ventajas de usar NavGraph:

---

**Visualización:** Puedes ver la estructura de navegación de forma gráfica en Android Studio.

**Safe Args:** Permite pasar argumentos de forma segura entre destinos.

**Deep linking:** Facilita la creación de enlaces profundos para acceder directamente a una pantalla específica.

**Integración con otras herramientas:** Se integra bien con otras herramientas de desarrollo, como Android Studio.

## Frameworks

Aunque en todos los casos se aplica el mismo flujo de trabajo general, la forma exacta de crear un host y un gráfico de navegación depende del framework de IU que uses.

- **Compose:** usa el elemento componible `NavHost`. Agrega un `NavGraph` con el [DSL de Kotlin](#). Puedes crear el gráfico de dos maneras:
  - **Como parte del NavHost:** construye el gráfico de navegación directamente como parte de agregar el `NavHost`.
  - **De manera programática:** usa el método `NavController.createGraph()` para crear un `NavGraph` y pasarlo directamente al `NavHost`.
- **Fragmentos:** Cuando uses fragmentos con el framework de IU de vistas, usa un `NavHostFragment` como host. Existen varias formas de crear un gráfico de navegación:
  - **De manera programática:** usa el DSL de Kotlin para crear un `NavGraph` y aplicarlo directamente en el `NavHostFragment`.
    - La función `createGraph()` que se usa con el DSL de Kotlin para fragmentos y Compose es la misma.
  - **XML:** escribe el host y el gráfico de navegación directamente en XML.
  - **Editor de Android Studio:** usa el editor de GUI de Android Studio para crear y ajustar tu gráfico como un archivo de recursos XML.

# Probando navigation component en entorno real

---

# NO!! al XML

---

Evidentemente evitaremos al máximo posible el uso de XML y de cualquier modelo tradicional.

Así que veremos como implementar la navegación de forma correcta y sencilla en Compose.



# Necesitaremos

---

1. Los **dataclass** serializables que van a representar las rutas (tambien pueden ser objetos)
2. Un **NavHost** para alojar el menu que usemos
3. Los **composables** que representan las pantallas de cada ruta
4. Un **NavController** para usarlo todo

# Estructura de carpetas

---

Yo voy a usar una estructura de carpetas que para mi es coherente, ni es la mas valida ni la mejor.

Siempre teneis que intentar estructurar el proyecto y la lógica de las carpetas en función de los criterios necesarios de cada proyecto.

# dataclass

---

Crearemos los dataclass que representen las rutas, usaremos dataclass porque permiten pasar parámetros a las rutas, si queremos una ruta si parámetros usaremos objects.

El archivo ha sido llamado **rutas** para que no os lieis pero lo suyo seria que hubiese un archivo por cada Dataclass para luego poder reutilizarlo en las consultas a las API o firebase.

Ya que realmente no son rutas, son clases, cosa distinta es que el NavHost lo use para las rutas

Android ▾

▾ app

> manifests

▾ kotlin+java

▾ com.example.patata

▾ navegacion

myNavHost.kt

rutas.kt

▾ pantallas

Pantallas.kt

> ui.theme

MainActivity.kt

> com.example.patata (androidTest)

> com.example.patata (test)

▾ res

> drawable

> mipmap

▾ values

</> colors.xml

</> strings.xml

MainActivity.kt

myNavHost.kt

Pantallas.kt

rutas.kt

```
1 package com.example.patata.navegacion
2
3 import kotlinx.serialization.Serializable
4
5 @Serializable
6 data class Home(val name: String = "Home")
7
8 @Serializable
9 data class Perfil(val name: String = "Perfil")
10
11 //Ejemplo con object
12
13 @Serializable
14 object Ajustes
```

## 2. NavHost

---

Crearemos un navhost aparte para poder reutilizarlo si queremos en otras activitys o donde sea.

Recibe un innerpadding (no obligatorio) para poder usar el Scaffold y un NavController porque necesita saber el controlador que va a usar.-

Podria no recibir nada y crear el propio NavController en el NavHost pero asi podemos reutilizarlo y pasarle un NavController desde donde queramos

Android ▾

▼ app

▸ manifests

▼ kotlin+java

▼ com.example.patata

▼ navegacion

myNavHost.kt

rutas.kt

▼ pantallas

Pantallas.kt

▸ ui.theme

MainActivity.kt

▸ com.example.patata (androidTest)

▸ com.example.patata (test)

▼ res

▸ drawable

▸ mipmap

▼ values

</> colors.xml

</> strings.xml

</> themes.xml

▸ xml

res (generated)

▼ Gradle Scripts

build.gradle.kts (Project: Patata)

build.gradle.kts (Module :app)

proguard-rules.pro (ProGuard Rules for ":app")

MainActivity.kt

myNavHost.kt

Pantallas.kt

rutas.kt

build.gradle.kts (Patata)

```
1 package com.example.patata.navigacion
2
3 import androidx.compose.foundation.layout.PaddingValues
4 import androidx.compose.runtime.Composable
5 import androidx.navigation.NavHostController
6 import androidx.navigation.compose.NavHost
7 import androidx.navigation.compose.composable
8 import androidx.navigation.compose.rememberNavController
9 import com.example.patata.pantallas.HomeScreen
10 import com.example.patata.pantallas.ProfileScreen
11
12
13 @Composable
14 fun MyNavHost(innerPAdding : PaddingValues, navController: NavHostController) {
15
16     NavHost(navController = navController, startDestination = "home") {
17         composable(route: "home") { HomeScreen(innerPAdding, navController) }
18         composable(route: "profile") { ProfileScreen(profile = "Perfil") }
19     }
20
21
22 }
```

## 3. Screens

---

Añadiremos las vistas para renderizar cada ruta, yo lo he hecho todo en un archivo pero podéis hacerlo en archivos individuales si quereis.

A Home screen le paso el NavController para que pueda hacer uso del mismo menú que usan todos.

## Android

### app

#### manifests

#### kotlin+java

#### com.example.patata

#### navegacion

#### myNavHost.kt

#### rutass.kt

#### pantallas

#### Pantallas.kt

#### ui.theme

#### MainActivity.kt

#### com.example.patata (androidTest)

#### com.example.patata (test)

#### res

#### drawable

#### mipmap

#### values

#### colors.xml

#### strings.xml

#### themes.xml

#### xml

#### res (generated)

#### Gradle Scripts

#### build.gradle.kts (Project: Patata)

#### build.gradle.kts (Module :app)

#### proguard-rules.pro (ProGuard Rules for ":app")

#### gradle.properties (Project Properties)

#### gradle-wrapper.properties (Gradle Version)

## MainActivity.kt

## myNavHost.kt

## Pantallas.kt

## rutass.kt

## build.gradle.kts (Patata)

```

7 import androidx.compose.material3.MaterialTheme
8 import androidx.compose.material3.Text
9 import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Modifier
11 import androidx.navigation.NavController
12
13 @Composable
14 fun ProfileScreen(
15     profile: String,
16 ) {
17     Text(text: "Profile for ${profile}")
18     Button(onClick = { }) {
19         Text(text: "Go to Friends List")
20     }
21 }
22
23 @Composable
24 fun HomeScreen(innerPadding: PaddingValues, navController: NavController) {
25     Column {
26         Text(
27             text = "Android",
28             modifier = Modifier.padding(innerPadding)
29         )
30         Button(onClick = {
31             navController.navigate(route: "profile")
32         }) {
33             Text(text = "Perfil")
34         }
35     }
36 }

```



## 4. NavController

---

Usaremos todo en una activity y crearemos un NavController para que sea común a todo el ambito y ciclo de vida de la activity y los composables que lo alojan.

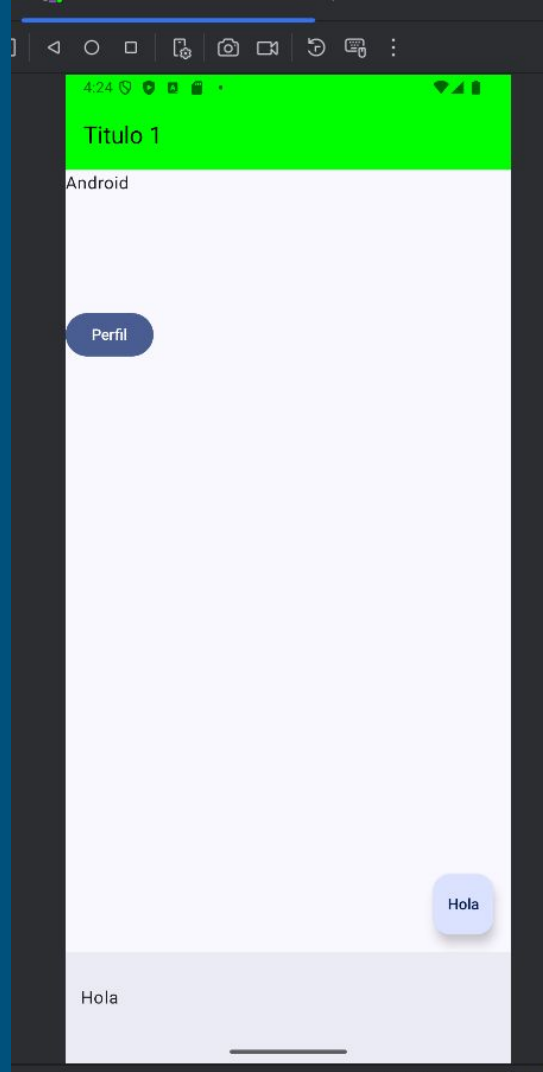
Luego le paso ese NavController al NavHost que hemos creado y lo meto dentro del “content” del componente principal de la activity que es el Scaffold asi hereda los margins y padding para que no se solape.

Podría haberlo metido en PatataTheme? Si, pero no lo he hecho.  
Sois libres de hacerlo como querais, esa es la magia de compose.

```

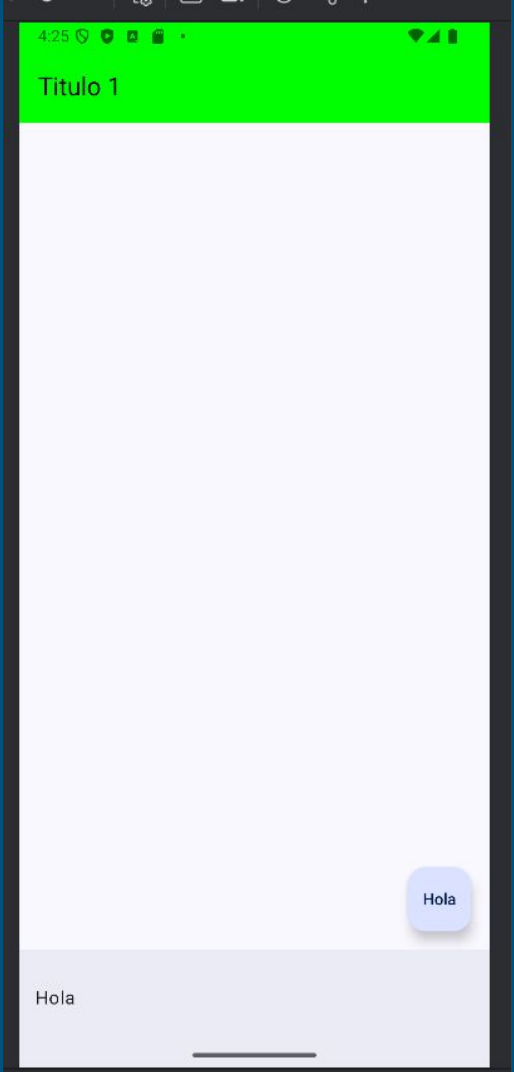
32 class MainActivity : ComponentActivity() {
33     override fun onCreate(savedInstanceState: Bundle?) {
34         super.onCreate(savedInstanceState)
35         enableEdgeToEdge()
36         setContent {
37             PatataTheme {
38                 val navController = rememberNavController()
39                 Scaffold(
40                     modifier = Modifier.fillMaxSize(),
41                     topBar = {
42                         TopAppBar(modifier = Modifier.background(Color.Cyan), title = { Text(text = "Titulo 1") },
43                             colors = TopAppBarColors(containerColor = Color.Green, scrolledContainerColor = Color.Cyan, navigationIconContentColor = Color.Cyan, titleContentColor = Color.Cyan))
44                     },
45                     bottomBar = {
46                         BottomAppBar(content = {
47                             Text(text = "Hola", modifier = Modifier.padding(10.dp))
48                         })
49                     },
50                     floatingActionButton = {
51                         androidx.compose.material3.FloatingActionButton(
52                             onClick = {
53                                 //navegamos a Perfil
54                                 navController.navigate(route = "profile")
55                             }
56                         ) {Text(text = "Hola")}
57                     }
58                 ) { innerPadding ->
59                     MyNavHost(innerPadding, navController)
60                 }
61             }
62         }
63     }
64 }

```



Al pulsar en el boton “hola”,  
ire a “Perfil”.

El boton lo usan ambas  
porque la activity está  
alojando todas las pantallas y  
por tanto el Scaffold que  
usan es comun



# Ejercicios intermedios

---

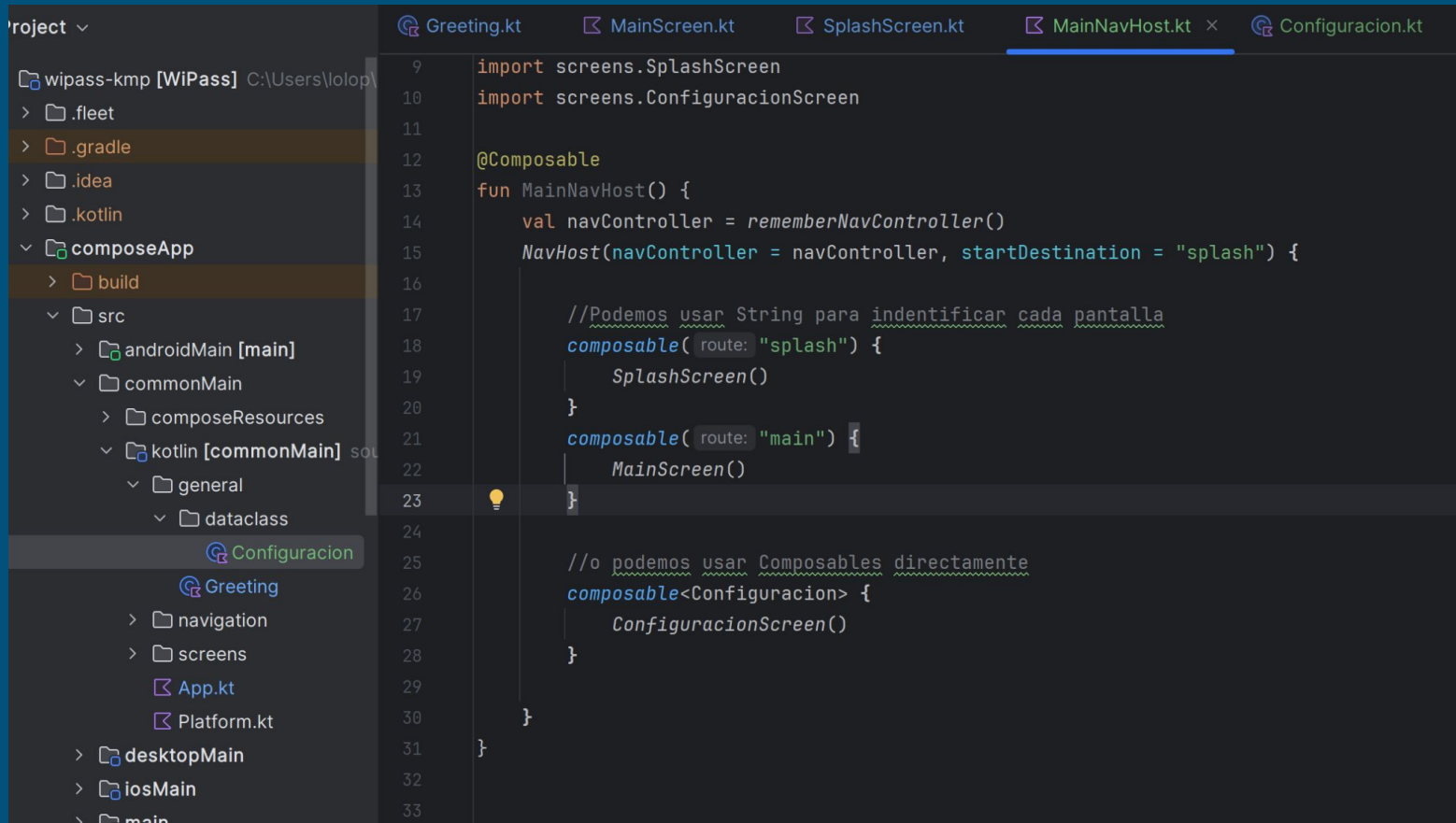
6. Formulario de registro: Crea un formulario de registro con campos para nombre, correo electrónico y contraseña, utilizando `ConstraintLayout` para posicionar los elementos de forma precisa.

7. Navegación entre pantallas: Implementa una navegación básica entre dos pantallas utilizando `Navigation Compose`. La primera pantalla mostrará una lista de elementos y al hacer clic en un elemento, se navegará a una pantalla de detalle.

# Mandando parámetros

---

# Podemos usar Strings o Composables



Para lo strings no es necesario tener la Dataclass, pero si la Screen.  
Crearemos la Dataclass para Configuración

Greeting.kt MainScreen.kt SplashScreen.kt MainNavHost.kt Configuracion.kt ×

```
package general.dataclass
```



```
data class Configuracion(val name : String = "configuracion")
```

# Vamos a implementar el Login

Para pasar navegacion a una screen podemos pasar por parametro el NavController o la función lambda de navigate().

Aqui usaremos una lambda.

```
//o podemos usar Composables directamente
composable<Configuracion> {
    ConfiguracionScreen()
}

//para pasar navegacion a una screen podemos pasar por parametro el NavController o la función lambda de navigate()
composable( route: "login") {
    LoginScreen { NavController.navigate( route: "main") }
}
```



De esta forma  
aceptamos una Lambda  
por parametro para  
usarla en nuestra  
Screen

```
@Composable
fun LoginScreen(navigateToHome: () -> Unit) {
    Scaffold {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "¡Bienvenido a la pantalla de login!",
                style = MaterialTheme.typography.h5
            )
            Spacer(modifier = Modifier.height(16.dp))
            Button(onClick = { navigateToHome() }) {
                Text(text: "Login")
            }
        }
    }
}
```

# Ventajas de pasar NavController o función

---

Hay diferencias clave entre pasar el NavController entero a tus composables (Screens) o solo pasar una función lambda para la navegación (navigate()).

# NavController

---

```
@Composable
fun MyScreen(navController: NavHostController) {
    // ...
    Button(onClick = { navController.navigate("otherScreen") }) {
        Text("Ir a otra pantalla")
    }
    // ...
}
```

# Ventajas:

---

**Flexibilidad total:** Tienes acceso a todas las funciones del NavController, como `navigate()`, `popBackStack()`, `currentBackStackEntry`, etc. Esto te da un control completo sobre la navegación.

**Funcionalidades avanzadas:** Puedes usar funcionalidades más complejas del NavController, como pasar argumentos a las pantallas, navegar a un gráfico de navegación específico, etc.

**Acceso al estado de la navegación:** Puedes observar el estado actual de la navegación (por ejemplo, la pantalla actual) directamente desde el NavController.

# Desventajas:

---

**Acoplamiento:** Tus composables se acoplan directamente a la implementación de la navegación (el NavController). Esto hace que sean menos reutilizables y más difíciles de probar de forma aislada.

**Responsabilidad única:** Un composable que solo debería mostrar UI ahora también tiene la responsabilidad de la navegación. Esto puede complicar el código y hacerlo menos legible.

**Pruebas:** Probar un composable que recibe el NavController puede ser más complejo, ya que necesitas mockear o proporcionar un NavController en tus pruebas.

**Complejidad:** Puede llevar a un código más complejo en las pantallas, especialmente si se usan muchas funciones del NavController.

# función lambda navigate()

---

```
@Composable
fun MyScreen(onNavigateToOtherScreen: () -> Unit) {
    // ...
    Button(onClick = onNavigateToOtherScreen) {
        Text("Ir a otra pantalla")
    }
    // ...
}
```

# Ventajas:

---

**Desacoplamiento:** Tus composables están desacoplados de la implementación de la navegación. Solo necesitan saber que hay una función para navegar, pero no cómo se realiza la navegación.

**Reutilización:** Tus composables son más reutilizables, ya que no dependen del NavController. Puedes usarlos en diferentes contextos de navegación.

**Responsabilidad única:** Tus composables se centran en la UI y delegan la navegación a un nivel superior.

**Pruebas:** Probar un composable que recibe una lambda es mucho más sencillo. Solo necesitas pasar una lambda de prueba.

**Simplicidad:** El código en las pantallas es más simple y legible.

# Desventajas:

---

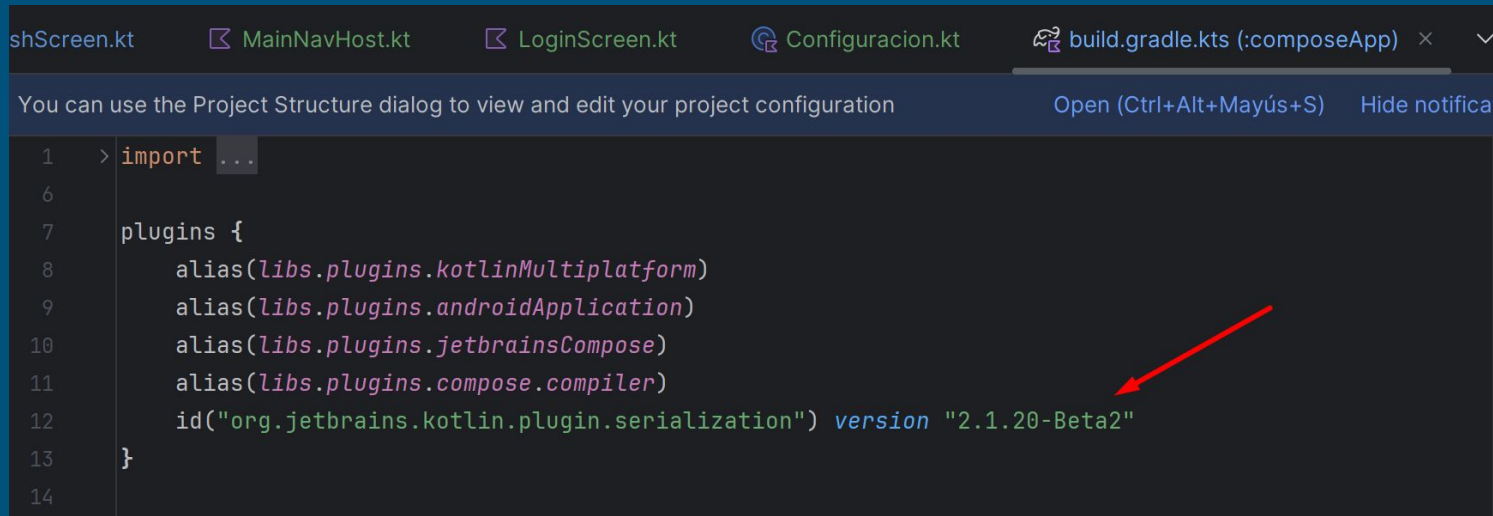
**Menos flexibilidad:** Solo tienes la capacidad de navegar a una nueva pantalla. Si necesitas funcionalidades más complejas del NavController, este enfoque puede ser limitante.

**Menos control:** Pierdes el control directo sobre el estado de la navegación.

**Boilerplate:** Puede generar un poco de código repetitivo si se necesitan muchas funciones de navegación diferentes.



# Librerías para Compose multiplatform



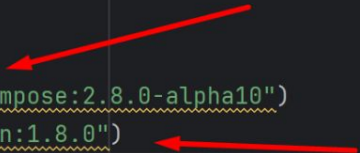
```
shScreen.kt  MainNavHost.kt  LoginScreen.kt  Configuracion.kt  build.gradle.kts (:composeApp) x v
You can use the Project Structure dialog to view and edit your project configuration  Open (Ctrl+Alt+Mayús+S)  Hide notifica

1  > import ...
2
3
4
5
6
7  plugins {
8      alias(libs.plugins.kotlinMultiplatform)
9      alias(libs.plugins.androidApplication)
10     alias(libs.plugins.jetbrainsCompose)
11     alias(libs.plugins.compose.compiler)
12     id("org.jetbrains.kotlin.plugin.serialization") version "2.1.20-Beta2"
13 }
14
```

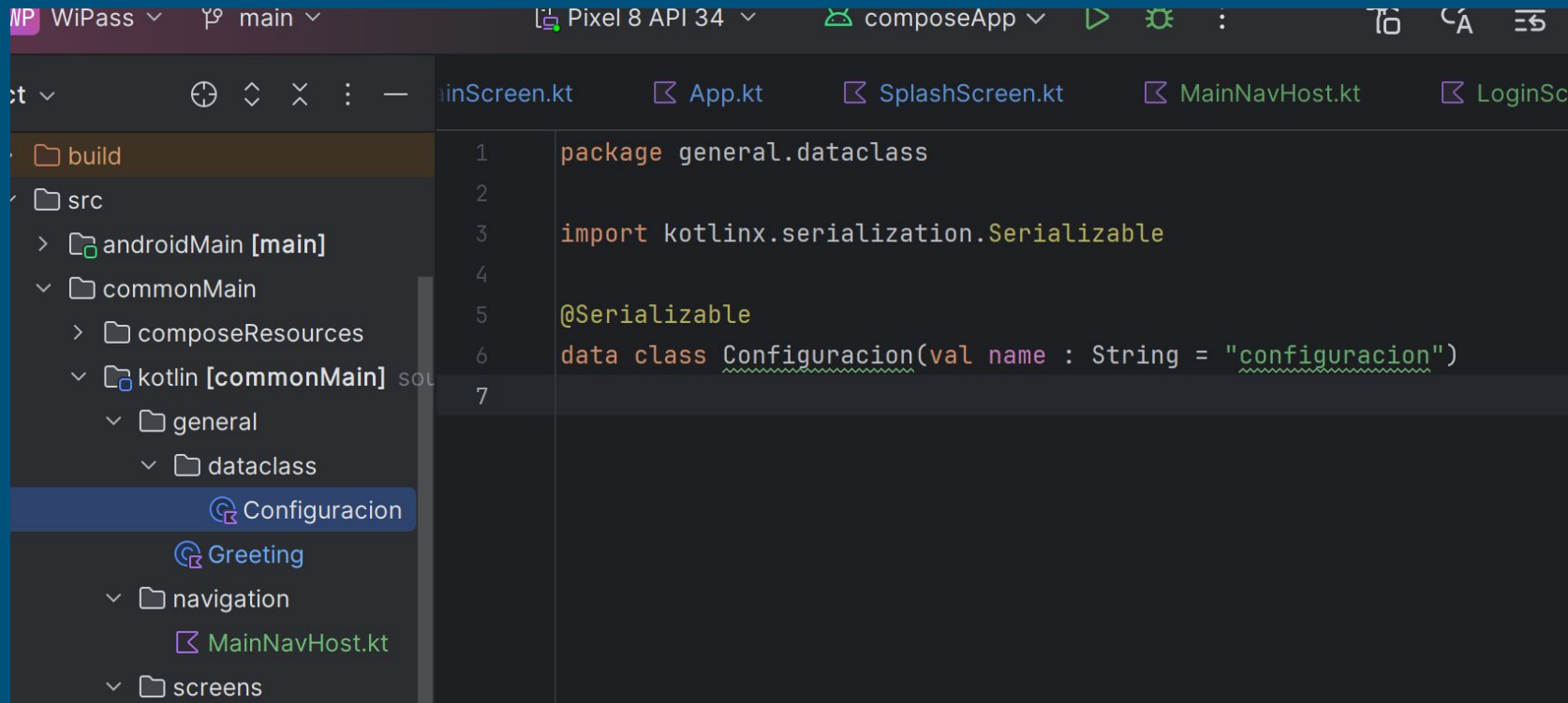
an use the Project Structure dialog to view and edit your project configuration

Open (Ctrl+Alt+Mayú+S) Hide notifica

```
kotlin {  
    sourceSets {  
        androidMain.dependencies {  
            implementation(libs.androidx.activity.compose)  
            implementation(libs.androidx.core.splashscreen)  
        }  
        commonMain.dependencies {  
            implementation(compose.runtime)  
            implementation(compose.foundation)  
            implementation(compose.material)  
            implementation(compose.ui)  
            implementation(compose.components.resources)  
            implementation(compose.components.uiToolingPreview)  
            implementation(dependencyNotation: "org.jetbrains.androidx.navigation:navigation-compose:2.8.0-alpha10")  
            implementation(dependencyNotation: "org.jetbrains.kotlinx:kotlinx-serialization-json:1.8.0")  
        }  
    }  
}
```

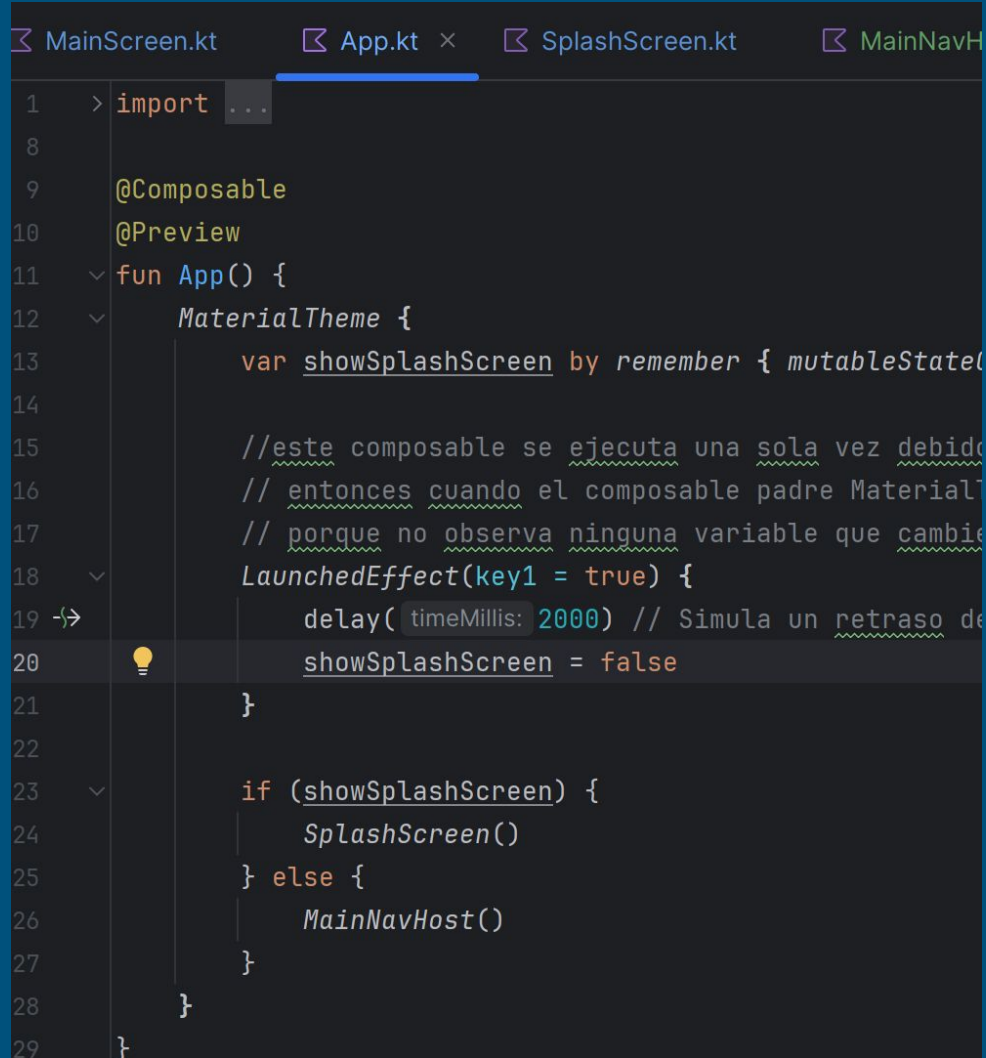


# Añadimos las dataclass para Configuración



# Incluimos el NavHost para que funcione todo

Como veréis hay muchas forma de hacerlo. Yo voy a usar un Composable llamado App() del que va a partir todo e incluiré el NavHost



```
1  > import ...
8
9  @Composable
10 @Preview
11 fun App() {
12     MaterialTheme {
13         var showSplashScreen by remember { mutableStateOf(true) }
14
15         //este composable se ejecuta una sola vez debido a que es @Preview
16         // entonces cuando el composable padre MaterialTheme se ejecuta
17         // porque no observa ninguna variable que cambie
18         LaunchedEffect(key1 = true) {
19             delay(timeMillis = 2000) // Simula un retraso de 2 segundos
20             showSplashScreen = false
21         }
22
23         if (showSplashScreen) {
24             SplashScreen()
25         } else {
26             MainNavHost()
27         }
28     }
29 }
```

# Vamos a pasar parametros a la Home desde Login

---

# Crearemos una dataclass para login

---

```
package general.dataclass

import kotlinx.serialization.Serializable

@Serializable
data class Login(val name : String = "login" )
```

# Modificamos la Screen de Login

De esta forma la LoginScreen será capaz de pasarle parámetros a la Home cuando le mande la función Lambda.

```
> import ...

@Composable
fun LoginScreen(navigateToHome: (username:String, password:String) -> Unit) {
    var username by remember { mutableStateOf( value: "" ) }
    var password by remember { mutableStateOf( value: "" ) }

    Scaffold {
        Column(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "¡Bienvenido a la pantalla de login!",
                style = MaterialTheme.typography.h5
            )
            TextField(
                value = "",
                onValueChange = { username = it },
                label = { Text( text: "Username" ) }
            )
            Spacer(modifier = Modifier.height(16.dp))
            TextField(
                value = "",
                onValueChange = { password = it },
                label = { Text( text: "Password" ) }
            )
            Spacer(modifier = Modifier.height(16.dp))
            Button(onClick = { navigateToHome(username, password) }) {
                Text( text: "Login" )
            }
        }
    }
}
```

# Cambiamos el NavHost

---

```
//para pasar navegacion a una screen podemos pasar por parametro el navController o la funcion lambda de navigate()  
composable<Login> {  
    LoginScreen { navController.navigate("main") }  
}
```



---

He cambiado el tipo de composable porque realmente Login no necesita ser un Dataclass

```
// para pasar navegacion a una screen podemos pasar por parametro el NavController o la funcion lambda de navigate()
composable(route: "login") {
    // LoginScreen { NavController.navigate("main") }

    // le pasamos el username y password a la siguiente pantalla junto a la funcion lambda de navigate()
    LoginScreen { username, password ->
        NavController.navigate(Home(username = username, password = password))
    }
}
```

# Para recuperar los datos en la otra Screen (Home)

---

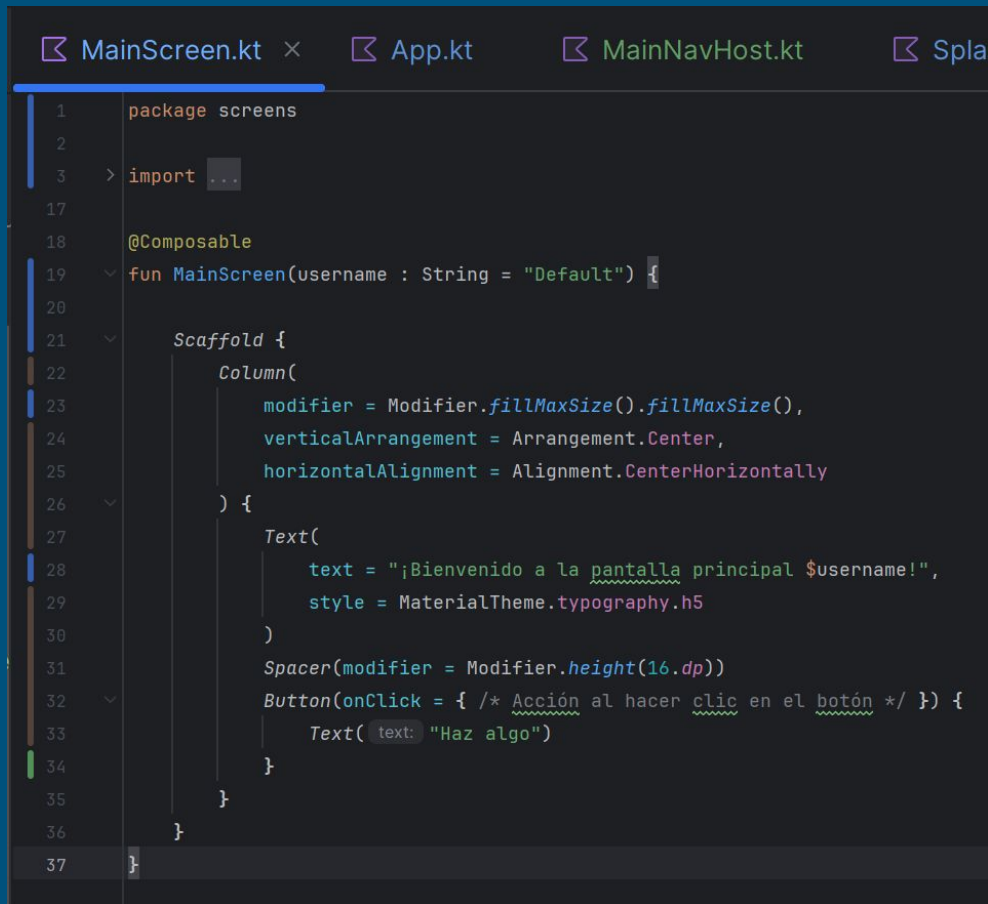
Tendremos que usar `backStackEntry` de navigation para recuperar las variables de la ruta.

Como el `$_REQUEST` de PHP

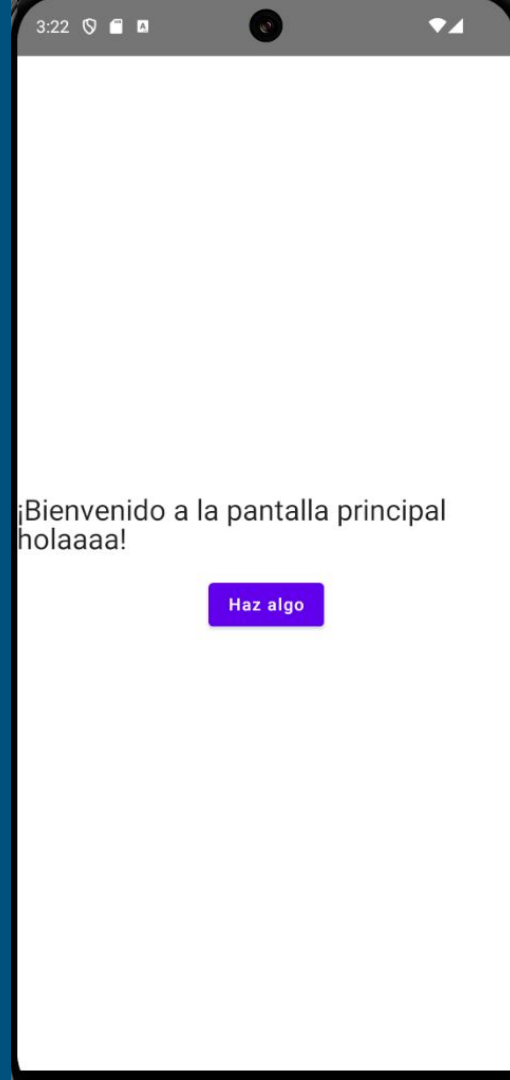
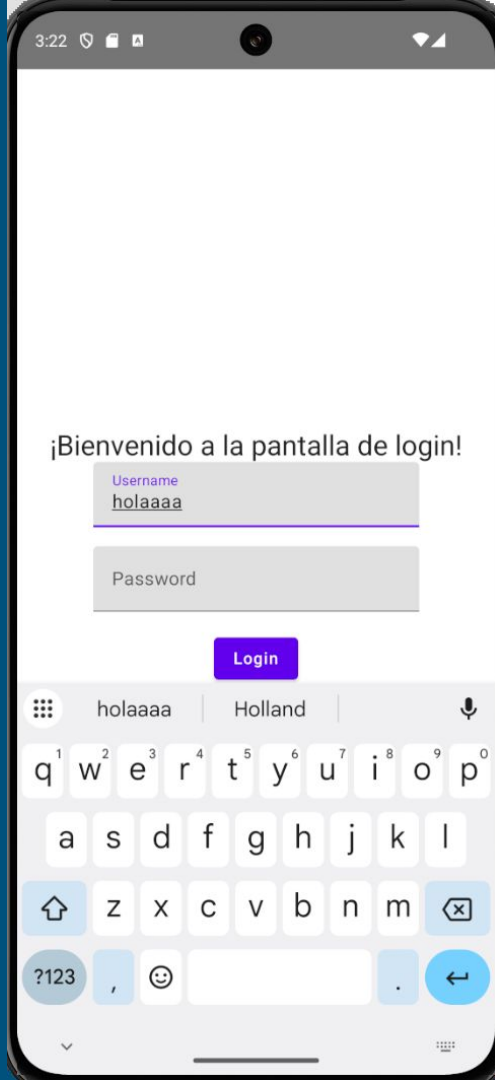
```
}  
  
composable<Home> { backStackEntry ->  
    //Recogemos los parametros de la navegacion como objeto Home  
    val home : Home = backStackEntry.toRoute()  
    MainScreen(home.username)  
}
```



# Nuestra Home o MainScreen



```
1 package screens
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18 @Composable
19 fun MainScreen(username : String = "Default") {
20
21     Scaffold {
22         Column(
23             modifier = Modifier.fillMaxSize().fillMaxSize(),
24             verticalArrangement = Arrangement.Center,
25             horizontalAlignment = Alignment.CenterHorizontally
26         ) {
27             Text(
28                 text = "¡Bienvenido a la pantalla principal $username!",
29                 style = MaterialTheme.typography.h5
30             )
31             Spacer(modifier = Modifier.height(16.dp))
32             Button(onClick = { /* Acción al hacer clic en el botón */ }) {
33                 Text(text: "Haz algo")
34             }
35         }
36     }
37 }
```



# Navegación hacia atrás

---

Vamos a añadir la funcionalidad de navegación hacia atrás en la pantalla de configuración

```
7 @Preview
8 @Composable
9 fun ConfiguracionScreen(navigateBack: () -> Unit) {
10     Column(
11         modifier = Modifier.fillMaxSize(),
12         verticalArrangement = Arrangement.Center,
13         horizontalAlignment = Alignment.CenterHorizontally
14     ) {
15         Text(
16             text = "¡Bienvenido a la pantalla de configuracion!",
17             style = MaterialTheme.typography.h5
18         )
19         Spacer(modifier = Modifier.height(16.dp))
20         Button(onClick = { navigateBack() }) {
21             Text(text: "Haz algo")
22         }
23     }
24 }
```

Esta función da un salto hacia atrás en la navegación en la pila de pantallas.

Hará que naveguemos una a una en la pila

```
gradlew  gradle.properties  MainScreen.kt  MainNavHost.kt  App.kt
16 fun MainNavHost() {
17     NavHost(navController = navController, startDestination = "login") {
26         composable(route: "login") {
31             navController.navigate(Home(username = username, password = password))
32         }
33     }
34     composable<Home> { backStackEntry ->
35         //Recogemos los parametros de la navegacion como objeto Home
36         val home : Home = backStackEntry.toRoute()
37         MainScreen(home.username)
38     }
39
40     //o podemos usar Composables directamente
41     composable<Configuracion> {
42         ConfiguracionScreen(){
43             //Le pasamos desde la pantalla de configuracion la funcion lambda para navigate
44             navController.popBackStack()
45         }
46     }
47
48 }
49
50 }
51
52
53
```

**@MainThread**  
**public open actual fun popBackStack(): Boolean**

Attempts to pop the controller's back stack. Analogous to when the user presses the system back button when the associated navigation host has focus.

Returns: true if the stack was popped at least once and the user has been navigated to another destination, false otherwise

**androidx.navigation.NavController**

Gradle: org.jetbrains.androidx.navigation...mBjA.klib

kotlin > navigation > MainNavHost.kt > MainN 44:45

## 2ª Forma

---

Si ahora en vez de ir hacia atrás queremos ir hacia una actividad, podríamos hacer `navController.navigate()` y eso sería correcto.

Pero si por ejemplo queremos hacer un logout no podemos navegar hacia logout porque el usuario podrá navegar hacia atrás y se “colaría” en una actividad o Screen que no queremos.



# Añadimos la función de logout

```
3 > import ...
17
18 @Composable
19 fun MainScreen(username : String = "Default", navigateLogout: () -> Unit) {
20     ⚡
21     Scaffold {
22         Column(
23             modifier = Modifier.fillMaxSize().fillMaxSize(),
24             verticalArrangement = Arrangement.Center,
25             horizontalAlignment = Alignment.CenterHorizontally
26         ) {
27             Text(
28                 text = "¡Bienvenido a la pantalla principal $username!",
29                 style = MaterialTheme.typography.h5
30             )
31             Spacer(modifier = Modifier.height(16.dp))
32             Button(onClick = { navigateLogout() }) {
33                 Text(text = "Logout")
34             }
35         }
36     }
37 }
```

# Y ahora le pasamos el logout desde el navhost

```
31         navController.navigate(Home(username = username, password = password))
32     }
33 }
34 composable<Home> { backStackEntry ->
35     //Recogemos los parametros de la navegacion como objeto Home
36     val home : Home = backStackEntry.toRoute()
37     MainScreen(home.username){
38         //Para hacer logout
39         navController.navigate( route: "login"){
40             popUpTo( route: "login")
41         }
42     }
43 }
44 }
45
46 //o podemos usar Composables directamente
47 composable<Configuracion> {
```

# popUpTo()

---

Esta función hará que naveguemos hasta la ruta pero limpiemos toda la pila de pantallas de la navegación hasta la que estemos navegando, es decir, si navegamos a Login, limpia todas las pantallas de la navegación menos las de Login.

Digo “las” porque si hemos navegado 30 veces al login, quedarán 30 pantallas de Login en la pila

# Problema

---

Al quedar todas las pantallas de Login en la pila si le doy hacia atras me volverá a cargar las mismas pantallas y eso está feo :(

Así que usaremos: **inclusive** = true

# Esto hará que limpie todas las pantallas incluida la que estamos llamando

---

```
}  
composable<Home> { backStackEntry ->  
    //Recogemos los parametros de la navegacion como objeto Home  
    val home : Home = backStackEntry.toRoute()  
    MainScreen(home.username){  
        //Para hacer logout  
        navController.navigate( route: "login"){  
            popUpTo( route: "login"){  
                inclusive = true  
            }  
        }  
    }  
}  
}
```

# navigateUp()

---

¿Qué hace? Intenta navegar "hacia arriba" en la jerarquía de navegación. Esto puede o no llevarte a la misma pantalla que popBackStack, dependiendo de cómo hayas configurado tu navegación.

Básicamente respeta la jerarquía de la navegación

# Diferencias clave

---

**Jerarquía:** `navigateUp` respeta la jerarquía de navegación, mientras que `popBackStack` simplemente retrocede en la pila.

**Comportamiento:** `popBackStack` siempre te lleva a la pantalla anterior en la pila, mientras que `navigateUp` puede llevarte a una pantalla diferente según tu configuración.

**Botón "Atrás":** `popBackStack` se asemeja más al comportamiento del botón "Atrás" del sistema.

# Navegación a una app externa (ej. abrir un PDF)

---

1. Se inicia una nueva actividad: La app externa (visor de PDF) se abre en una nueva actividad, separada de la pila de actividades de tu app Compose.
2. Tu app Compose queda en segundo plano: Tu app Compose permanece en segundo plano, pero la actividad que abrió el PDF sigue en la pila



# Comportamiento de popBackStack y navigateUp después de abrir una app externa

---

Imaginemos que tenemos implementado la navegación hacia atrás en la app y llamamos a un visor de pdf, si uso el `popBackStack()` en mi app, navegará hacia atrás pero al pdf, sin embargo si uso `navigateUp()` navegaré al flujo natural de la app, es decir, a la pantalla anterior que ha llamado a la pantalla que llama al pdf.

Ejemplo para ilustrar la diferencia (dentro de tu app):

---

Imagina esta secuencia:

Pantalla A -> Pantalla B -> Pantalla C

`popBackStack()` desde la pantalla C te lleva a la pantalla B.

`navigateUp()` desde la pantalla C también te lleva a la pantalla B (en este caso coinciden).

Pero si tienes una jerarquía diferente:

Pantalla A -> Pantalla B -> Pantalla C (y la pantalla B tiene un "hijo" que es la pantalla D)

`popBackStack()` desde la pantalla C te lleva a la pantalla B.

`navigateUp()` desde la pantalla C te lleva a la pantalla D (porque está "más arriba" en la jerarquía).

# En resumen:

---

- Un "hijo" en navegación se refiere a una pantalla que se considera una subsección o extensión de otra pantalla (el "padre").
- Esta relación se define en el gráfico de navegación.
- `navigateUp()` tiene en cuenta esta relación para navegar "hacia arriba" en la jerarquía.

